

RA1 – Programación de aplicaciones con varios procesos

Parte teórica

1. ¿Qué es la programación concurrente?

- a) La ejecución de un único proceso en varios núcleos.
 - b) La ejecución de varias tareas que se alternan o ejecutan simultáneamente.
 - c) La ejecución distribuida en varios equipos.
 - d) La ejecución secuencial de un único hilo.
-

2. ¿Qué diferencia principal hay entre la programación paralela y la distribuida?

- a) La paralela usa varios equipos; la distribuida, un solo equipo.
 - b) La paralela ocurre en un solo equipo, la distribuida en varios equipos conectados por red.
 - c) Ambas son equivalentes.
 - d) La paralela es más lenta.
-

3. En Java, ¿qué clase se usa para ejecutar un proceso del sistema operativo?

- a) `Thread`
 - b) `Runtime` o `ProcessBuilder`
 - c) `ExecutorService`
 - d) `SystemProcess`
-

4. ¿Qué método permite esperar a que un proceso finalice su ejecución?

- a) `join()`
 - b) `wait()`
 - c) `exit()`
 - d) `waitFor()`
-

5. Un proceso en ejecución tiene su propio:

- a) PID y memoria independiente.
 - b) PID compartido con otros procesos.
 - c) Espacio de memoria compartido por defecto.
 - d) Ninguno de los anteriores.
-

6. ¿Qué método del objeto `Process` permite leer su salida estándar?

- a) `getOutputStream()`

- b) `getInputStream()`
 - c) `getErrorStream()`
 - d) `readOutput()`
-

7. ¿Qué es un subproceso en Java?

- a) Un método estático.
 - b) Un hilo dentro de otro hilo.
 - c) Un proceso hijo lanzado desde otro.
 - d) Un proceso que ejecuta código nativo.
-

8. ¿Qué característica NO pertenece a la programación concurrente?

- a) Compartición de recursos.
 - b) Sincronización entre tareas.
 - c) Ejecución secuencial estricta.
 - d) Comunicación entre procesos.
-

9. En Java, para ejecutar “notepad.exe” desde tu aplicación:

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");
```

```
Process p = pb.start();
```

¿Qué hará este código?

- a) Fallará al compilar.
 - b) Abrirá el bloc de notas.
 - c) Esperará la finalización automática.
 - d) Cerrará todos los procesos.
-

10. ¿Qué función cumple `exitValue()` en un proceso Java?

- a) Devuelve el PID del proceso.
 - b) Devuelve el código de salida del proceso.
 - c) Devuelve la salida estándar.
 - d) Detiene el proceso.
-

Parte práctica (código)

11.

```
ProcessBuilder pb = new ProcessBuilder("calc");
```

```
pb.start();
```

¿Qué hace este código?

- a) Lanza la calculadora del sistema.
 - b) Crea un hilo.
 - c) Compila un archivo.
 - d) Lanza una excepción obligatoriamente.
-

12.

```
Process p = Runtime.getRuntime().exec("ping www.google.com");
```

¿Qué devuelve este método?

- a) Un `int`.
 - b) Un objeto `Process`.
 - c) Un `Thread`.
 - d) Nada.
-

13.

¿Cuál es el método correcto para capturar la salida del proceso?

```
BufferedReader br = new BufferedReader(new InputStreamReader(_____  
));
```

- a) `p.getOutputStream()`
 - b) `p.getInputStream()`
 - c) `p.getErrorStream()`
 - d) `System.in`
-

14.

Si quieres esperar que el proceso termine antes de continuar:

```
int exit = p._____( );
```

- a) `join()`
 - b) `wait()`
 - c) `waitFor()`
 - d) `stop()`
-

15.

¿Qué sucede si intentas acceder a un archivo mientras otro proceso lo usa sin sincronización?

- a) Se bloquea el sistema.
 - b) Se genera una condición de carrera o error de acceso.
 - c) El archivo se sobrescribe.
 - d) No ocurre nada.
-

16.

La clase `ProcessBuilder` permite:

- a) Crear hilos concurrentes.
 - b) Crear y configurar procesos del sistema.
 - c) Programar tareas temporizadas.
 - d) Sincronizar hilos.
-

17.

¿Qué método del proceso devuelve un flujo para enviarle datos al proceso hijo?

- a) `getInputStream()`
 - b) `getOutputStream()`
 - c) `getErrorStream()`
 - d) `getWriter()`
-

18.

¿Qué implica documentar una aplicación concurrente?

- a) Incluir solo los nombres de los hilos.
 - b) Explicar el flujo, dependencias y sincronización.
 - c) Escribir comentarios automáticos.
 - d) Ninguna.
-

19.

¿Qué tipo de aplicaciones usan varios procesos?

- a) Juegos simples.
- b) Compiladores, navegadores, servidores.
- c) Calculadoras básicas.

d) Ninguna.

20.

¿Cuál de estas tareas no requiere paralelismo real?

- a) Procesar grandes volúmenes de datos en varios núcleos.
 - b) Descargar archivos desde distintos servidores.
 - c) Ejecutar un único bucle for.
 - d) Renderizar gráficos 3D.
-



RA2 – Programación con varios hilos



Parte teórica

21.

¿Cuál es la diferencia entre proceso e hilo?

- a) Un hilo tiene su propia memoria; un proceso la comparte.
 - b) Un hilo comparte memoria con otros del mismo proceso.
 - c) Los procesos no existen en Java.
 - d) Son lo mismo.
-

22.

Para crear un hilo en Java hay que:

- a) Instanciar un objeto **Thread** y llamar a **start()**.
 - b) Llamar al método **run()** directamente.
 - c) Extender **Runnable** y llamar a **execute()**.
 - d) Crear una clase **Hilo**.
-

23.

¿Qué estado adopta un hilo tras llamar a **sleep()**?

- a) **RUNNABLE**
 - b) **BLOCKED**
 - c) **TIMED_WAITING**
 - d) **NEW**
-

24.

¿Qué palabra clave evita condiciones de carrera?

- a) **protected**
- b) **synchronized**

- c) `static`
 - d) `volatile`
-

25.

¿Qué método espera a que otro hilo termine antes de continuar?

- a) `sleep()`
 - b) `wait()`
 - c) `join()`
 - d) `stop()`
-

26.

Un hilo que nunca obtiene CPU porque otros tienen más prioridad sufre:

- a) Deadlock
 - b) Starvation (inanición)
 - c) Race condition
 - d) Overhead
-

27.

¿Cuál NO es un estado posible de un hilo?

- a) NEW
 - b) RUNNABLE
 - c) STOPPED
 - d) TERMINATED
-

28.

¿Qué clase del paquete `java.util.concurrent` gestiona un grupo de hilos?

- a) `ThreadPool`
 - b) `ExecutorService`
 - c) `RunnableService`
 - d) `ThreadGroup`
-

29.

¿Qué tipo devuelve `submit()` en un `ExecutorService`?

- a) `Future`
 - b) `Thread`
 - c) `Runnable`
 - d) `Process`
-

30.

¿Qué se usa para establecer prioridad en un hilo?

- a) `Thread.setPriority()`
 - b) `Thread.priority = ...`
 - c) `Thread.setLevel()`
 - d) `Thread.setImportance()`
-

Parte práctica (código)

31.

```
class Hilo extends Thread {  
  
    public void run() {  
  
        System.out.println("Hola desde el hilo");  
  
    }  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        new Hilo().start();  
  
    }  
  
}
```

¿Qué mostrará en consola?

- a) Nada.
 - b) "Hola desde el hilo".
 - c) Error de compilación.
 - d) Excepción.
-

32.

¿Qué pasará si en lugar de `start()` se llama directamente a `run()`?

- a) Se ejecutará en un nuevo hilo.
- b) Se ejecutará en el hilo principal.

- c) Dará error.
 - d) Se ejecutará dos veces.
-

33.

```
Thread t = new Thread(() -> System.out.println("Ejecutando"));

t.start();

t.join();

System.out.println("Fin");
```

¿Qué imprime?

- a) Fin \n Ejecutando
 - b) Ejecutando \n Fin
 - c) Nada
 - d) Error
-

34.

```
synchronized void incrementar() { contador++; }
```

¿Qué garantiza **synchronized**?

- a) Que varios hilos modifiquen a la vez.
 - b) Que solo un hilo acceda a la vez al método.
 - c) Que el contador no cambie.
 - d) Ninguna.
-

35.

¿Cuál de estos mecanismos pertenece a **java.util.concurrent**?

- a) **Thread.sleep()**
 - b) **Semaphore**
 - c) **Runnable**
 - d) **ProcessBuilder**
-

36.

¿Cuál de estos ejemplos produce una condición de carrera?

- a) Varios hilos incrementan una variable compartida sin sincronización.
 - b) Un hilo esperando con `sleep()`.
 - c) Un hilo leyendo datos inmutables.
 - d) Un solo hilo modificando datos.
-

37.

¿Qué método de `Thread` detiene temporalmente la ejecución del hilo actual?

- a) `wait()`
 - b) `pause()`
 - c) `sleep()`
 - d) `delay()`
-

38.

¿Qué clase sirve para ejecutar tareas que devuelven un resultado?

- a) `Runnable`
 - b) `Callable`
 - c) `Thread`
 - d) `Executor`
-

39.

En una aplicación multihilo, ¿qué ocurre si dos hilos esperan mutuamente un recurso?

- a) Starvation
 - b) Deadlock
 - c) Race condition
 - d) Overload
-

40.

¿Cuál de las siguientes opciones describe mejor el contexto de ejecución de un hilo?

- a) Comparte la pila con otros hilos.
- b) Tiene su propia pila y registros, pero comparte heap.
- c) Tiene su propio heap y pila.
- d) Usa la memoria del sistema operativo directamente.